

SANTÉGAB

Logiciel de gestion du parcours médical des patients

Appel à candidatures KIMBA CONNECT — Gabon

-
1. Dossier de candidature
 2. Documentation technique
 3. Guide utilisateur
 4. Déploiement et maintenance
 5. Sécurité et données personnelles

Dossier documentaire complet — Juillet 2026

Sommaire

1. Dossier de candidature

1. Présentation
2. Couverture du cahier des charges
3. Périmètre de la démonstration
4. Déroulé de démonstration (20 minutes)
5. Calendrier de mise en œuvre
6. Travaux préalables à une mise en production
7. Documentation remise
8. Réversibilité

2. Documentation technique

1. Vue d'ensemble
2. Choix techniques
3. Organisation du code
4. Modèle de données
5. Règles transverses
6. Communication client-serveur
7. Modularité et périmètre
8. Sécurité
9. Évolutions techniques identifiées

3. Guide utilisateur

Se connecter

Ce que voit tout le monde

 Médecin

 Infirmier

 Laborantin

 Radiologue

 Personnel administratif / accueil

 Administrateur

Questions fréquentes

4. Déploiement et maintenance

1.1 Prérequis

1.2 Variables d'environnement

1.3 Ordre de mise en place de la base

1.4 Déploiement de l'application

1.5 Contrôles après mise en service

1.6 Retour arrière

2.1 Maintenance corrective

2.2 Maintenance préventive

2.3 Sauvegardes

2.4 Faire évoluer l'application

2.5 Pièges connus

2.6 Compétences requises

2.7 Réversibilité

5. Sécurité et données personnelles

1. Résumé pour le jury

2. Chiffrement des données

3. Authentification et contrôle d'accès

4. Cloisonnement entre établissements

5. Traçabilité

6. Protection des données personnelles et rétention

7. Points à traiter avant une mise en production réelle

8. Ce que cette note ne couvre pas

Dossier de candidature

Objet : logiciel de gestion du parcours médical des patients **Version :** juillet 2026

1. Présentation

SANTÉGAB est une application web de gestion du parcours médical, développée pour le contexte sanitaire gabonais. Elle couvre la prise en charge du patient de son enregistrement à la restitution de ses résultats d'examens : dossier médical, consultations, actes, prescriptions, rendez-vous, examens et traçabilité des accès.

L'application est fonctionnelle et démontrable en conditions réelles, avec un jeu de données représentatif : un établissement, neuf profils d'utilisateurs, des patients gabonais, des consultations et des examens.

Ce qui distingue la proposition

Un carnet de santé accessible sans compte. Un QR Code généré depuis le dossier ouvre, sur le téléphone du patient, une page présentant son groupe sanguin, ses allergies, ses antécédents et ses derniers résultats. Le code expire au bout de 24 heures et chaque accès est journalisé. En situation d'urgence, un soignant accède à l'essentiel sans identifiants.

Des antécédents qui suivent le patient. Allergies et traitements chroniques sont rattachés à la personne, pas à l'établissement. Un patient pris en charge dans une autre structure du réseau conserve ces informations. Les traitements en cours apparaissent en tête du dossier.

Une traçabilité qui couvre aussi les lectures. Le journal enregistre les créations et modifications, mais également **qui a ouvert quel dossier patient**. C'est l'exigence la plus souvent négligée en matière de données de santé.

Un contrôle des accès à deux niveaux. Les droits sont vérifiés à l'affichage et de nouveau côté serveur à chaque écriture : contourner l'interface ne donne accès à rien.

2. Couverture du cahier des charges

§	Exigence	État
5.1	Dossier patient : identité, contact, groupe sanguin, allergies	✓
5.1	Traitements en cours	✓
5.2	Consultations et prescriptions	✓
5.3	Examens biologiques, radiologiques et spécialisés	✓
5.4	Résultats : saisie, dépôt de fichiers, archivage	✓
5.5	Actes médicaux : professionnel, date, service	✓
5.6	Antécédents structurés	✓
5.7	Prise de rendez-vous et calendrier	✓
5.8	Gestion documentaire	✓
5.9	Tableaux de bord et statistiques	✓
6	Profils utilisateurs	✓ 9 profils (5 demandés)
7	Architecture modulaire	✓
7	Authentification et gestion des accès	✓
7	Chiffrement	✓ TLS + chiffrement au repos
7	API ouverte	✗ non couvert
8	Traçabilité des consultations de dossier	✓
9	Interopérabilité	✗ non couvert
10	Livrables documentaires	✓

Les deux exigences non couvertes

Nous les signalons plutôt que de les contourner.

SANTÉGAB n'expose **aucune API publique** : les échanges entre l'interface et le serveur passent par des appels internes, sans route HTTP exposée. Ce choix réduit la surface d'attaque, mais interdit en l'état toute interconnexion avec un système tiers.

L'architecture y est préparée : chaque domaine métier est isolé dans son propre fichier de fonctions serveur, avec des signatures explicites. **Ouvrir une API REST ou FHIR consisterait à ajouter une couche d'exposition au-dessus de l'existant, sans réécrire la logique métier.** Nous estimons cette évolution à six à huit semaines, à engager dès qu'une norme d'échange nationale sera arrêtée.

3. Périmètre de la démonstration

SANTÉGAB comporte des modules de gestion administrative et financière (facturation, comptabilité, pharmacie, hospitalisations) **qui ne relèvent pas du cahier des charges**. Ils sont masqués pour la candidature.

Ce masquage est piloté par une constante unique dans le code. Le rétablir est une opération d'une ligne. Cette mécanique illustre concrètement la modularité demandée au CDC §7 : **un domaine fonctionnel s'active ou se retire par configuration**, sans intervention sur le reste de l'application.

Ces modules restent disponibles si un établissement souhaite en disposer ultérieurement.

4. Déroulé de démonstration (20 minutes)

1 — Accueil : enregistrer un patient (3 min)

Profil administratif. Créer un patient : identité, groupe sanguin, allergies. Le numéro de dossier est attribué automatiquement.

2 — Prendre un rendez-vous (3 min)

Menu Rendez-vous, calendrier de la semaine. Créer un rendez-vous, puis **tenter d'en créer un second sur le même créneau avec le même médecin** : le logiciel refuse et nomme le patient déjà positionné.

3 — La consultation (5 min)

Profil médecin. Ouvrir le dossier, enregistrer une consultation : nature de l'acte, motif, constantes, diagnostic, prescriptions.

Onglet Antécédents : ajouter un traitement chronique actif. **Il remonte immédiatement dans l'encart « Traitements en cours »** en tête du dossier.

4 — Les examens (3 min)

Prescrire un examen de laboratoire. *Profil laborantin* : saisir le résultat et joindre le compte rendu. Le résultat apparaît dans le dossier du patient.

5 — Le carnet de santé (3 min)

Générer le QR Code depuis la fiche patient. **Le scanner avec un téléphone du jury** : la page s'ouvre sans identifiants et présente groupe sanguin, allergies, antécédents et derniers résultats. Rappeler l'expiration à 24 heures et la journalisation de chaque accès.

6 — Traçabilité et droits (3 min)

Profil administrateur. Journal d'audit : filtrer sur « Consultation » pour montrer **qui a ouvert quel dossier**. Exporter en CSV.

Puis le module Permissions : retirer un droit à un profil, se reconnecter avec ce profil, constater que le module a disparu du menu.

5. Calendrier de mise en œuvre

Phase	Durée	Contenu
1 — Installation	2 semaines	Environnement, base, comptes, personnalisation
2 — Reprise des données	3 semaines	Import du fichier patients existant
3 — Formation	2 semaines	Une session par profil, supports remis
4 — Pilote	4 semaines	Un service, accompagnement rapproché
5 — Généralisation	3 semaines	Extension à l'établissement
Total	14 semaines	

Les phases 1 et 2 peuvent être menées en parallèle.

Points de vigilance annoncés

La reprise des données existantes est le poste le plus incertain. Sa durée dépend entièrement de l'état des fichiers actuels : un tableur structuré se reprend en quelques jours, des dossiers papier demandent une saisie que seul l'établissement peut planifier.

La formation conditionne l'adoption. Un logiciel de dossier médical ne produit ses effets que si les soignants le renseignent. Nous prévoyons une session par profil et un accompagnement pendant la phase pilote.

6. Travaux préalables à une mise en production

Par honnêteté technique, ces points sont identifiés et documentés dans *SECURITE.md* :

1. Définir le secret de signature des QR Codes (`QR_TOKEN_SECRET`)
2. Activer la sécurité au niveau des lignes sur les tables métier restantes
3. Réaliser un exercice de restauration de sauvegarde
4. Automatiser la politique de conservation des données
5. Faire valider les durées de conservation par l'autorité compétente

Aucun ne remet en cause l'architecture. Ils relèvent de la préparation à l'exploitation réelle.

7. Documentation remise

Document	Contenu
<i>ARCHITECTURE.md</i>	Choix techniques, modèle de données, modularité
<i>GUIDE-UTILISATEUR.md</i>	Mode d'emploi par profil
<i>EXPLOITATION.md</i>	Déploiement, maintenance, réversibilité
<i>SECURITE.md</i>	Chiffrement, accès, traçabilité, données personnelles

8. Réversibilité

Le socle technique est intégralement libre : PostgreSQL, Next.js, Prisma. Les données s'exportent aux formats standard, le code source est remis.

Aucun verrou technique ne s'oppose à une reprise par un autre prestataire, ni à une réinstallation sur une infrastructure nationale.

Documentation technique

Destinataire : dossier de candidature KIMBA CONNECT (CDC §10) **Version :** juillet 2026

1. Vue d'ensemble

SANTÉGAB est une application web de gestion du parcours médical des patients, conçue pour un usage multi-établissements. Une seule instance dessert plusieurs structures de santé, chacune ne voyant que ses propres données.

L'application est **monolithique et modulaire** : un seul déploiement, mais des domaines fonctionnels indépendants les uns des autres, activables ou masquables sans toucher au reste du code.

2. Choix techniques

Couche	Technologie	Raison du choix
Interface	Next.js 16 (App Router)	Rendu serveur : les données médicales ne transitent pas inutilement vers le navigateur
Composants	Shadcn UI + Tailwind CSS 4	Composants accessibles, pas de dépendance à une bibliothèque propriétaire
Accès aux données	Prisma 7	Schéma typé, requêtes vérifiées à la compilation
Base de données	PostgreSQL 17 (Supabase)	Base relationnelle éprouvée, adaptée aux données structurées de santé
Authentification	Supabase Auth	Gestion des mots de passe déléguée, jamais stockés par l'application
Fichiers	Supabase Storage	Résultats d'examens, comptes rendus
Hébergement	Vercel	Déploiement continu, HTTPS automatique

Langage unique : TypeScript du navigateur jusqu'à la base. Un même modèle de données est partagé entre l'interface et le serveur, ce qui supprime toute une catégorie d'erreurs d'intégration.

3. Organisation du code

app/	
├─ (auth)/login/	Connexion
├─ carnet/[token]/	Carnet de santé public (QR Code, sans compte)
└─ dashboard/	
├─ patients/	Dossier patient, antécédents, QR Code
├─ consultations/	Consultations, actes, prescriptions
├─ rendez-vous/	Calendrier médical
├─ laboratory/	Examens biologiques + catalogue
├─ imaging/	Imagerie médicale
├─ services/	Services médicaux
├─ stats/	Tableaux de bord
├─ audit/	Journal de traçabilité
├─ users/	Comptes utilisateurs
└─ permissions/	Rôles et droits
components/	Composants d'interface, groupés par domaine
lib/	Règles transverses (voir §5)
prisma/schema.prisma	Modèle de données
scripts/	Scripts SQL à exécuter manuellement
docs/	Documentation

Chaque domaine suit la même structure : une page, un fichier d'actions serveur, des composants dédiés. Un développeur qui connaît un module les connaît tous.

4. Modèle de données

27 tables, organisées en six domaines.

Identité et structure

Hospital · Utilisateur · Service · Permission · RolePersonnalise

Dossier patient

Patient · PatientHospital · AntecedentMedical

PatientHospital est la table pivot : **un patient peut être suivi par plusieurs établissements**. Ses données d'identité sont uniques et partagées ; son rattachement, son assurance et son médecin traitant sont propres à chaque structure.

Parcours de soins

Consultation · Prescription · RendezVous

Examens

ExamenLabo · ExamenCatalogue · ExamenLaboExamen · ExamenImagerie

Traçabilité

`AuditTrail` · `QrToken` · `AuditLogCarnet`

Hors périmètre KIMBA (présents mais masqués — voir §7)

`Facture` · `LigneFacture` · `EcritureComptable` · `ArticleStock` · `MouvementStock` · `Chambre` · `Lit` · `Hospitalisation` · `LigneHospitalisation`

Règle structurante

Toute table métier porte une colonne `hospital_id`. C'est le pivot du cloisonnement entre établissements.

Une exception documentée : `AntecedentMedical` est lisible depuis tout établissement. Une allergie ou un traitement chronique appartient au patient, pas à la clinique qui l'a consigné — le masquer priverait le carnet de santé de son utilité en urgence. L'établissement d'origine reste enregistré et demeure seul habilité à modifier sa saisie.

5. Règles transverses

Fichier	Rôle
<code>lib/prisma.ts</code>	Connexion unique à la base (évite d'épuiser le pool)
<code>lib/audit.ts</code>	Journalisation — appelée après chaque action
<code>lib/permissions.ts</code>	Modules et libellés (utilisable côté navigateur)
<code>lib/permissions.server.ts</code>	Vérification des droits (serveur uniquement)
<code>lib/withPermission.ts</code>	Garde d'accès aux pages
<code>lib/kimba-scope.ts</code>	Périmètre fonctionnel activé (voir §7)

La séparation entre `permissions.ts` et `permissions.server.ts` est volontaire : elle empêche qu'un composant navigateur importe par erreur du code d'accès à la base.

6. Communication client-serveur

L'application n'expose **aucune API REST publique**. Les échanges passent par les *Server Actions* de Next.js : des fonctions serveur appelées directement depuis l'interface, sans route HTTP exposée.

Conséquence positive : la surface d'attaque est réduite. Il n'existe aucun point d'entrée interrogeable depuis l'extérieur, donc rien à protéger par clé d'API ou limitation de débit.

Limite assumée : aucune interconnexion avec un système tiers n'est possible en l'état (exigences CDC §7 « API ouverte » et §9 « Interopérabilité »). L'architecture y est cependant préparée : chaque domaine métier est

isolé dans son propre fichier d'actions, avec des signatures explicites. Exposer ces fonctions via des routes REST ou FHIR consisterait à ajouter une couche d'exposition, sans réécrire la logique métier.

Modèle imposé à chaque action serveur

```
export async function monAction(hospitalId, utilisateurId, utilisateurNom, data) {
  await verifierPermissionAction(hospitalId, "MODULE", "peut_creer"); // 1. droits
  const resultat = await prisma.model.create({ ... });                // 2. métier
  await enregistrerAudit({ ... });                                    // 3. trace
  return resultat;
}
```

Les trois étapes sont systématiques. La vérification des droits a lieu côté serveur : contourner l'interface ne donne aucun accès supplémentaire.

7. Modularité et périmètre

SANTÉGAB dépasse le cahier des charges KIMBA sur plusieurs domaines (facturation, comptabilité, pharmacie, hospitalisations). Ces modules sont **masqués, non supprimés**.

Le fichier `lib/kimba-scope.ts` contient une constante unique :

```
export const MASQUAGE_KIMBA_ACTIF: boolean = true;
```

Elle pilote le masquage des entrées de menu, l'accès direct par URL et les éléments financiers imbriqués dans les écrans conservés. La passer à `false` réactive l'ensemble, sans autre modification.

C'est la démonstration concrète de la modularité exigée au CDC §7 : un domaine fonctionnel s'active ou se retire par configuration.

8. Sécurité

Traitée en détail dans **SECURITE.md** : chiffrement, authentification, contrôle d'accès à deux niveaux, cloisonnement, traçabilité, conservation des données, et points à traiter avant mise en production.

9. Évolutions techniques identifiées

Par ordre de priorité :

1. Activer la sécurité au niveau des lignes (RLS) sur les tables métier restantes, pour doubler le cloisonnement applicatif d'une garantie en base.
2. Automatiser la politique de conservation des données.

3. Exposer une API REST ou FHIR si l'interopérabilité devient nécessaire.
4. Mettre en place des tests automatisés — l'application n'en comporte pas à ce jour, la vérification repose sur le contrôle de types et la compilation.

Guide utilisateur

Destinataire : dossier de candidature KIMBA CONNECT (CDC §10) **Version :** juillet 2026

Ce guide décrit ce que chaque profil peut faire. Les écrans visibles varient selon les droits : un utilisateur ne voit dans le menu que les modules qui le concernent.

Se connecter

1. Ouvrir l'adresse de l'application.
2. Saisir son adresse professionnelle et son mot de passe.
3. Le tableau de bord s'affiche, adapté au profil.

En cas d'oubli du mot de passe, l'administrateur de l'établissement le réinitialise depuis le module Utilisateurs.

Ce que voit tout le monde

Le tableau de bord affiche l'activité du jour : nombre de patients enregistrés, consultations du jour et du mois, patients en salle d'attente, et la liste des **rendez-vous de la journée**.



Médecin

Accès : patients, consultations, rendez-vous, laboratoire, imagerie, statistiques.

Consulter un dossier patient

Menu **Patients**, rechercher par nom, prénom ou numéro de dossier, puis cliquer sur la ligne. Le dossier s'ouvre en quatre onglets : consultations, prescriptions, antécédents, et (hors périmètre KIMBA) factures.

Chaque ouverture de dossier est enregistrée dans le journal de traçabilité : nom du praticien, patient concerné, date et heure.

Enregistrer une consultation

Depuis la fiche patient, bouton **Nouvelle consultation** — le patient est alors déjà sélectionné. Ou depuis le menu **Consultations**, bouton **Nouvelle consultation**.

Renseigner :

- la **nature de l'acte** : consultation médicale ou soin ;
- le motif (obligatoire), le service, le statut ;
- les constantes : tension, poids, taille, température ;
- le diagnostic et les observations ;
- les **prescriptions**, ajoutées une par une (médicament, dosage, fréquence, durée).

Renseigner les antécédents

Onglet **Antécédents** du dossier, bouton **Ajouter**. Choisir le type : pathologie, hospitalisation, chirurgie, allergie ou traitement chronique.

Les traitements chroniques marqués « actif » remontent automatiquement en tête du dossier, dans un encart « Traitements en cours ». C'est l'information qu'un confrère doit voir en premier.

Les antécédents suivent le patient d'un établissement à l'autre. Ceux saisis ailleurs sont visibles mais non modifiables — ils portent la mention « Saisi par un autre établissement ».

Planifier un rendez-vous

Menu **Rendez-vous**. Le calendrier affiche la semaine ; le nombre de rendez-vous figure sous chaque jour. Cliquer sur un jour pour voir le détail.

Bouton **Nouveau** : patient, médecin, service, date, heure, durée, motif.

Le logiciel refuse un créneau déjà occupé par le même médecin et indique le patient concerné. Les doubles réservations sont impossibles.

Sur chaque rendez-vous : **Modifier** (report, changement de praticien) et les statuts **Confirmer, Honoré, Absent, Annuler**.

Prescrire un examen

Menus **Laboratoire** et **Imagerie**, bouton de création. Le résultat est rattaché au dossier du patient dès sa validation par le service concerné.

Infirmier

Accès : patients (lecture), consultations (lecture), rendez-vous (création et modification), laboratoire et imagerie (lecture).

L'infirmier consulte les dossiers et les prescriptions, mais ne crée pas de consultation médicale. Il **gère les rendez-vous** : planification, confirmation, enregistrement des présences et des absences.

Les soins réalisés sont enregistrés par le médecin ou l'administratif avec la nature d'acte « Soin ».

Laborantin

Accès : laboratoire (création, modification), patients et consultations (lecture).

1. Menu **Laboratoire** : la liste des examens prescrits s'affiche.
2. Ouvrir l'examen à traiter.
3. Saisir les résultats, ou **joindre le compte rendu** en PDF.
4. Valider : le résultat devient visible dans le dossier du patient.

Le **catalogue des examens** (familles, libellés, tarifs) est administré depuis le sous-menu dédié, réservé aux administrateurs.

Radiologie

Accès : imagerie (création, modification), patients et consultations (lecture).

Fonctionnement identique au laborantin, sur le module **Imagerie** : enregistrement du compte rendu et dépôt des fichiers d'images.

Personnel administratif / accueil

Accès : patients (création, modification), rendez-vous (tous droits), consultations (lecture).

C'est le profil **gestionnaire des rendez-vous** : il dispose seul du droit de suppression sur ce module.

Enregistrer un nouveau patient

Menu **Patients**, bouton **Nouveau patient**. Renseigner l'identité, les coordonnées, le groupe sanguin, les allergies connues et, le cas échéant, l'organisme d'assurance et le taux de couverture.

Un **numéro de dossier** est attribué automatiquement.

Générer le carnet de santé QR Code

Depuis la fiche patient, bouton **QR Code**. Un code est produit, **valable 24 heures**. Le patient le présente depuis son téléphone : la page s'ouvre sans compte et affiche son groupe sanguin, ses allergies, ses antécédents, ses dernières consultations et ses résultats d'examens.

Chaque accès au carnet est journalisé. Le code peut être désactivé à tout moment, et expire seul au bout de 24 heures.

Administrateur

Accès : tout, sans restriction.

Gérer les comptes

Menu **Utilisateurs** : création, modification, désactivation. Chaque compte reçoit un rôle qui détermine ses accès.

Ajuster les droits

Menu **Permissions**. Un tableau croise les modules et les quatre droits (voir, créer, modifier, supprimer). Les modifications sont enregistrées immédiatement.

Des **rôles personnalisés** peuvent être créés pour les métiers qui ne correspondent à aucun profil standard — sage-femme, aide-soignant, directeur médical — puis assignés aux utilisateurs.

Consulter le journal de traçabilité

Menu **Journal d'audit**. Chaque ligne indique l'auteur, la date et l'heure, l'action, le module et l'élément concerné.

Filtres disponibles : utilisateur, type d'action, module, période. **Export CSV** pour archivage ou transmission lors d'un contrôle.

Le filtre « **Consultation** » isole les lectures de dossier : qui a ouvert quel dossier patient, et quand.

Administrer les services

Menu **Services** : création des services médicaux (cardiologie, pédiatrie...) avec un code couleur repris dans les plannings et les consultations.

Questions fréquentes

Un module a disparu de mon menu. Les modules s'affichent selon les droits. Contacter l'administrateur.

Le calendrier refuse mon rendez-vous. Le médecin a déjà un rendez-vous qui chevauche ce créneau. Le message indique le patient concerné : choisir un autre horaire ou un autre praticien.

Je ne peux pas modifier un antécédent. Il a été saisi par un autre établissement. Il reste consultable, mais seule la structure qui l'a enregistré peut le corriger.

Le QR Code du patient ne fonctionne plus. Sa validité est de 24 heures. En générer un nouveau depuis la fiche patient.

Déploiement et maintenance

Destinataire : dossier de candidature KIMBA CONNECT (CDC §10) **Version** : juillet 2026

PARTIE 1 — DÉPLOIEMENT

1.1 Prérequis

Élément	Détail
Compte Supabase	Base PostgreSQL + authentification + stockage
Compte Vercel	Hébergement de l'application
Nom de domaine	Optionnel, à rattacher à Vercel
Node.js	Version 20 ou supérieure (développement uniquement)

Aucun serveur à administrer : les deux services sont infogérés.

1.2 Variables d'environnement

À définir dans Vercel (*Settings* → *Environment Variables*) **avant** le premier déploiement :

```
NEXT_PUBLIC_SUPABASE_URL=      # URL du projet Supabase
NEXT_PUBLIC_SUPABASE_ANON_KEY=  # Clé publique
SUPABASE_SERVICE_ROLE_KEY=     # Clé de service – SECRÈTE
DATABASE_URL=                  # Pooler, port 6543, pgbouncer=true
MIGRATE_DATABASE_URL=          # Pooler, port 5432, mode session
QR_TOKEN_SECRET=               # 32+ caractères aléatoires
```

 **QR_TOKEN_SECRET est obligatoire.** En son absence, l'application refuse de générer un QR Code en production. C'est délibéré : le secret de repli figure dans le code source, donc il est public.

Générer une valeur : `openssl rand -base64 32`

 **SUPABASE_SERVICE_ROLE_KEY contourne toutes les règles de sécurité de la base.** Elle ne doit jamais apparaître dans le code ni côté navigateur. Seules les variables préfixées `NEXT_PUBLIC_` sont exposées au client.

1.3 Ordre de mise en place de la base

Les scripts sont exécutés **manuellement** dans Supabase → SQL Editor, dans cet ordre :

Ordre	Script	Objet
1	<code>npx prisma db push</code>	Crée le schéma initial
2	<code>npx prisma db seed</code>	Jeu de données de démonstration (optionnel)
3	<code>scripts/catalogue_labos.sql</code>	Catalogue des examens
4	<code>scripts/phase1_rendez_vous.sql</code>	Module Rendez-vous
5	<code>scripts/phase2_antecedents_actes.sql</code>	Antécédents structurés et actes

⚠ **Le script de la Phase 1 comporte une instruction `ALTER TYPE ... ADD VALUE` à exécuter SEULE**, en dehors de tout bloc. PostgreSQL l'interdit à l'intérieur d'une transaction, et l'éditeur Supabase en ouvre une lorsqu'on exécute plusieurs instructions d'un coup. La marche à suivre est rappelée en tête du fichier.

Tous les scripts sont **idempotents** : les relancer ne casse rien.

1.4 Déploiement de l'application

1. Rattacher le dépôt Git au projet Vercel.
2. Vérifier les variables d'environnement.
3. Déployer. Vercel exécute `prisma generate && next build`.
4. Chaque envoi sur la branche principale redéploie automatiquement.

1.5 Contrôles après mise en service

- ☐ Connexion avec un compte de test
- ☐ Création d'un patient, puis d'une consultation
- ☐ Prise d'un rendez-vous, et **vérification du refus d'un créneau occupé**
- ☐ Ajout d'un antécédent, puis contrôle de l'encart « Traitements en cours »
- ☐ Génération d'un QR Code et ouverture depuis un téléphone
- ☐ Dépôt d'un résultat d'examen (contrôle du stockage de fichiers)
- ☐ Journal d'audit : présence des actions réalisées
- ☐ Connexion avec un profil non administrateur : vérifier que les modules non autorisés sont bien absents du menu

1.6 Retour arrière

Vercel conserve les déploiements précédents : un retour à la version antérieure se fait en une action depuis l'interface, sans reconstruction.

Les scripts SQL comportent chacun une section de retour arrière, commentée en fin de fichier.

PARTIE 2 — MAINTENANCE

2.1 Maintenance corrective

Gravité	Exemple	Prise en charge visée
Bloquante	Application inaccessible, connexion impossible	4 heures ouvrées
Majeure	Un module inutilisable, perte de données	1 jour ouvré
Mineure	Défaut d'affichage, libellé erroné	Prochaine version

Points de contrôle en cas d'incident :

1. **Vercel** → **Logs** : erreurs applicatives
2. **Supabase** → **Logs** : erreurs de base de données
3. **Journal d'audit** : dernières actions avant l'incident

2.2 Maintenance préventive

Chaque mois

- Contrôler le volume de la base et l'espace de stockage occupé
- Parcourir le journal d'audit à la recherche d'accès inhabituels
- Vérifier que les sauvegardes automatiques se déroulent correctement

Chaque trimestre

- Mettre à jour les dépendances (`npm outdated` , puis `npm update`)
- **Tester une restauration de sauvegarde** sur un projet séparé
- Revoir les comptes utilisateurs : désactiver les départs

Chaque année

- Revue des droits par profil
- Contrôle de l'application de la politique de conservation
- Vérification des durées auprès de l'autorité compétente

2.3 Sauvegardes

Assurées par Supabase : sauvegarde quotidienne automatique, restauration à un instant donné selon le plan souscrit.

Aucune restauration n'a été testée à ce jour. Un exercice complet est à réaliser avant toute mise en service réelle : une sauvegarde jamais restaurée n'est pas une sauvegarde.

2.4 Faire évoluer l'application

Ajouter un module

1. Ajouter le modèle dans `prisma/schema.prisma`, puis `npx prisma generate`
2. Ajouter le module dans `lib/permissions.ts` (`MODULES` + `MODULE_LABELS`)
3. Créer `app/dashboard/<module>/actions.ts` et `page.tsx`
4. Ajouter l'entrée dans `components/layout/Sidebar.tsx`
5. Si une valeur est ajoutée à l'énumération `ModuleAction`, **compléter** `MODULE_CONFIG` dans `components/audit/AuditList.tsx` — sinon la compilation échoue
6. Fournir le script SQL : création de table **et insertion des droits pour chaque rôle** (sans quoi le module reste invisible pour tous sauf les administrateurs)

Réactiver les modules masqués

Passer `MASQUAGE_KIMBA_ACTIF` à `false` dans `lib/kimba-scope.ts`. Facturation, comptabilité, pharmacie, hospitalisations et chambres réapparaissent. Aucune autre modification.

2.5 Pièges connus

Symptôme	Cause	Correction
<code>Cannot read properties of undefined (reading 'findMany')</code>	<code>prisma generate</code> lancé pendant que le serveur tournait	Redémarrer le serveur — recharger la page ne suffit pas
<code>ALTER TYPE ... cannot run inside a transaction block</code>	Script SQL exécuté d'un seul bloc	Exécuter l'instruction <code>ALTER TYPE</code> seule
Nouveau module invisible sauf pour l'administrateur	Droits absents en base	Exécuter le script d'insertion des permissions
<code>Module not found: Can't resolve 'dns'</code>	<code>permissions.server.ts</code> importé côté navigateur	N'importer que <code>permissions.ts</code> dans un composant client

Le fichier `CLAUDE.md` à la racine tient la liste complète et à jour.

2.6 Compétences requises

Un développeur web maîtrisant TypeScript et React peut reprendre le projet. Les conventions sont documentées dans `CLAUDE.md` et appliquées de façon homogène : commentaires en français, structure identique d'un module à l'autre.

Aucune compétence en administration de serveurs n'est nécessaire : hébergement et base de données sont infogérés.

2.7 Réversibilité

Le client reste propriétaire de ses données :

- **Export de la base** : `pg_dump` depuis Supabase, format PostgreSQL standard
- **Export des fichiers** : téléchargement depuis Supabase Storage
- **Code source** : dépôt Git, aucune dépendance propriétaire

Le socle technique (PostgreSQL, Next.js, Prisma) est libre. Une reprise par un autre prestataire, ou une réinstallation sur une infrastructure nationale, ne se heurte à aucun verrou technique.

Sécurité et données personnelles

Destinataire : dossier de candidature KIMBA CONNECT (CDC §7 et §8) **Version** : Phase 4 — juillet 2026
Périmètre : modules du cahier des charges KIMBA (dossier patient, consultations, examens, rendez-vous, antécédents, statistiques, audit)

1. Résumé pour le jury

Exigence CDC	État	Nature
Chiffrement des flux (TLS)	✓	Infrastructure (Vercel + Supabase)
Chiffrement au repos	✓	Infrastructure (Supabase / AWS)
Authentification	✓	Implémenté (Supabase Auth)
Gestion des accès par profil	✓	Implémenté (9 rôles + permissions granulaires)
Cloisonnement multi-établissements	⚠	Implémenté au niveau applicatif — voir §4
Traçabilité des écritures	✓	Implémenté (audit trail complet)
Traçabilité des lectures de dossier	✓	Implémenté en Phase 3
Politique de rétention	⚠	Définie ci-dessous, non automatisée

Les points marqués ⚠ sont documentés sans détour : ils correspondent à des choix d'architecture assumés, pas à des oublis.

2. Chiffrement des données

2.1 En transit

Toutes les communications sont chiffrées par TLS, sans exception :

- **Navigateur** → **application** : HTTPS imposé par Vercel, avec redirection automatique de HTTP vers HTTPS et en-tête HSTS.
- **Application** → **base de données** : connexion PostgreSQL en SSL. *Vérifié le 27/07/2026 : le serveur répond `ssl = on`.*
- **Application** → **stockage de fichiers** : API Supabase Storage en HTTPS.

2.2 Au repos

Le chiffrement au repos est assuré par l'infrastructure Supabase, hébergée sur AWS : volumes de base de données et sauvegardes chiffrés en AES-256.

Précision honnête : il s'agit d'une garantie contractuelle du prestataire d'hébergement, non d'un mécanisme développé dans SANTÉGAB. Elle doit être confirmée auprès de Supabase pour le plan souscrit au moment du déploiement. Aucun chiffrement applicatif supplémentaire (au niveau colonne) n'est mis en œuvre : les données médicales sont lisibles par quiconque dispose d'un accès administrateur légitime à la base.

2.3 Mots de passe

Les mots de passe ne transitent ni ne sont stockés par SANTÉGAB. L'authentification est entièrement déléguée à Supabase Auth, qui applique un hachage bcrypt. La base applicative ne contient aucun champ de mot de passe.

Environnement technique vérifié : PostgreSQL 17.6.

3. Authentification et contrôle d'accès

3.1 Sessions

Authentification par Supabase Auth, session portée par un cookie sécurisé. Un middleware vérifie la session sur l'ensemble des routes `/dashboard`. Toute requête sans session valide est redirigée vers la page de connexion.

3.2 Permissions

Le contrôle d'accès est appliqué à **deux niveaux**, ce qui évite qu'une faille d'interface n'ouvre l'accès aux données :

1. **À l'affichage** — la page serveur appelle `withPermission(module, action)`, qui redirige si le droit est absent. La barre de navigation masque en outre les modules non autorisés.
2. **À l'écriture** — chaque Server Action appelle `verifierPermissionAction()` avant toute opération. Un appel forgé directement vers l'action est donc rejeté même si l'interface a été contournée.

Neuf profils sont disponibles (le CDC en demandait cinq), avec quatre droits par module : voir, créer, modifier, supprimer. Des rôles personnalisés peuvent être créés par établissement.

Limite assumée : les rôles ADMIN et SUPER_ADMIN contournent intégralement le système de permissions. C'est un choix explicite, mais il implique que le compte administrateur d'un établissement accède à toutes les données de cet établissement, sans restriction possible.

3.3 Carnet de santé par QR Code

La page `/carnet/[token]` est **publique par conception** : elle doit rester consultable en urgence, sans compte. Sa sécurité repose sur trois barrières :

1. Le jeton est une chaîne imprévisible, générée aléatoirement.
2. Il doit exister dans la table `qr_tokens` — un jeton fabriqué de toutes pièces est rejeté, la présence en base étant la véritable garantie.
3. Il expire automatiquement au bout de **24 heures**, et peut être désactivé manuellement à tout moment (`est_actif`).

Chaque accès au carnet est journalisé (identifiant du jeton, adresse IP, navigateur, horodatage).

Correctif apporté en Phase 4 : le secret de signature des jetons disposait d'une valeur de repli inscrite dans le code source, donc publique. Elle n'était pas exploitable — la signature n'est jamais vérifiée, seule la présence en base fait foi — mais constituait un piège pour l'avenir. L'application **refuse désormais de démarrer en production** si la variable `QR_TOKEN_SECRET` n'est pas définie.

Action requise avant mise en production : définir `QR_TOKEN_SECRET` avec une valeur aléatoire d'au moins 32 caractères.

4. Cloisonnement entre établissements

SANTÉGAB est multi-établissements : chaque structure ne doit accéder qu'à ses propres données. Ce cloisonnement s'appuie sur la colonne `hospital_id`, présente sur toutes les tables métier.

Le cloisonnement est appliqué au niveau applicatif. Chaque requête filtre sur l'identifiant de l'établissement de l'utilisateur connecté, lequel provient toujours de la session authentifiée — jamais d'un paramètre d'URL, qui serait manipulable.

Point de vigilance documenté : au 27/07/2026, la sécurité au niveau des lignes (Row Level Security) de PostgreSQL n'est activée que sur **6 des 27 tables** du schéma public. La base ne constitue donc pas un second rempart pour la majorité des tables : une requête applicative qui omettrait le filtre `hospital_id` exposerait les données d'autres établissements.

Cette contrainte est compensée par une règle de développement stricte et documentée (fichier `CLAUDE.md`), appliquée systématiquement dans le code. **Recommandation** : activer la RLS sur l'ensemble des tables métier avant tout déploiement accueillant plusieurs établissements réels.

Exception documentée : les antécédents médicaux

Les antécédents structurés (allergies, pathologies, traitements chroniques) sont **volontairement lisibles depuis tout établissement**. Une allergie est une caractéristique du patient, pas la propriété d'une clinique : la masquer priverait le carnet de santé de son intérêt en situation d'urgence.

L'établissement d'origine reste enregistré, et **seul lui peut modifier ou supprimer** l'antécédent qu'il a saisi.

5. Traçabilité

Le journal d'audit enregistre, pour chaque événement : l'auteur, la date et l'heure, le type d'action, le module, l'entité concernée, l'adresse IP et le navigateur.

Sont tracées : les créations, modifications et suppressions de toutes les entités métier ; les connexions ; les générations et accès de QR Code ; les exports ; et, depuis la Phase 3, **les consultations de dossier patient**.

Ne sont pas tracées : l'ouverture des listes (patients, consultations, examens). Seul l'accès à un dossier nominatif est journalisé.

Une fenêtre de 30 minutes évite qu'une même personne consultant le même dossier génère des dizaines de lignes identiques — la page étant rechargée à chaque saisie. Le journal reste ainsi exploitable lors d'un contrôle.

Le journal est **consultable et exportable en CSV** par les administrateurs, avec filtres par utilisateur, type d'action, module et période.

Limite assumée : le journal est stocké dans la même base que les données métier. Un accès administrateur à la base permettrait d'en altérer le contenu. Une conservation sur un support distinct et en écriture seule serait nécessaire pour une valeur probante opposable.

6. Protection des données personnelles et rétention

6.1 Nature des données traitées

SANTÉGAB traite des **données de santé**, catégorie sensible par nature : identité, coordonnées, groupe sanguin, allergies, antécédents, diagnostics, prescriptions, résultats d'examens et documents médicaux.

6.2 Minimisation

Aucune donnée n'est collectée au-delà de ce qu'exige le suivi du parcours de soins. Aucun traceur publicitaire, aucun outil de mesure d'audience tiers, aucune transmission de données à un tiers ne sont mis en œuvre.

6.3 Durée de conservation proposée

Donnée	Durée proposée	Justification
Dossier patient et actes médicaux	20 ans après le dernier contact	Pratique hospitalière courante
Documents d'examens (fichiers)	Identique au dossier	Indissociables du dossier
Journal d'audit	3 ans	Besoin de contrôle et de preuve
Jetons QR Code	24 heures	Expiration technique automatique
Comptes utilisateurs désactivés	1 an après le départ	Traçabilité des actes passés

À ce jour, ces durées ne sont pas appliquées automatiquement. Aucune purge programmée n'existe : toutes les données sont conservées sans limite. La mise en œuvre suppose une tâche planifiée d'anonymisation ou de suppression, à développer, et une validation juridique des durées auprès de l'autorité gabonaise compétente.

6.4 Droits des personnes

La suppression d'un patient est possible depuis l'interface et retire en cascade ses consultations, examens et documents associés. L'opération est tracée dans le journal d'audit.

Le droit d'accès est matérialisé par le carnet de santé QR Code, que le patient peut consulter sur son propre téléphone.

Non couvert à ce jour : l'export des données d'un patient dans un format structuré réutilisable (droit à la portabilité), et le recueil formalisé du consentement.

6.5 Sauvegardes

Les sauvegardes sont assurées par Supabase (sauvegardes quotidiennes automatiques, restauration à un instant donné selon le plan souscrit). **Aucune procédure de restauration n'a été testée à ce jour** — un exercice de restauration est recommandé avant toute mise en service réelle.

7. Points à traiter avant une mise en production réelle

Par ordre de priorité :

1. **Définir** `QR_TOKEN_SECRET` — sans quoi l'application refusera de démarrer.
2. **Activer la RLS** sur les 21 tables métier qui en sont dépourvues, afin de disposer d'un second rempart au cloisonnement.
3. **Tester une restauration** de sauvegarde de bout en bout.
4. **Automatiser la politique de rétention** décrite au §6.3.
5. **Externaliser le journal d'audit** vers un stockage en écriture seule.
6. **Faire valider les durées de conservation** par l'autorité compétente.

8. Ce que cette note ne couvre pas

Par souci d'exactitude, les éléments suivants n'ont **pas** été réalisés et ne doivent pas être présentés comme acquis :

- Aucun test d'intrusion ni audit de sécurité externe.
- Aucune analyse d'impact relative à la protection des données (AIPD).
- Aucune certification d'hébergeur de données de santé.

- Aucun test de charge ni de continuité d'activité.